CS 202: Project - Online Food Ordering System

Spring 2026

Deadline
Project 1: **31th of May 2026, 11:55 PM**

Project groups will remain the same as in the previous project. Students should continue working with their group members in the worksheet: CS202- Spring 2026 Groups.
Note 1: The **deadline** is very **strict** and it will **NOT** be changed under any conditions. Late submissions will not be accepted.
Note 2: Demo sessions will be held after the submission deadline. The day, time, format, and access details of the demo sessions will be announced later. Students are required to present their work in approximately 15 minutes and answer questions about their implementation..
Reminder: On the Internet there exist similar projects; copying and submitting any of this work, or relying solely on AI-generated solutions, will result in a Penalty (-100) without any further discussion or consideration.

TAs' Email Address:

- Tuğçe Özgirgin: tugce.ozgirgin@ozu.edu.tr

1 Description

You will design and implement an Online Food Ordering System, similar to real-world platforms such as Yemeksepeti, GetirYemek, Uber Eats, or DoorDash. The goal is to create a full-stack system that supports restaurant browsing, menu management, food ordering, order tracking, ratings, and basic user/restaurant operations by using database technologies and web programming.

Users should be able to register, log in, browse restaurants and menus, add items to a cart, place orders, track order status, and rate restaurants after completed orders. Restaurant managers should be able to create and manage restaurant profiles, organize menu items, define discounts or coupons, handle incoming orders, and view monthly sales summaries.

The system should also include additional structures such as menu categories and coupons to make the project more detailed and realistic. For example, menu items may be grouped under categories such as starters, main dishes, desserts, and drinks, while restaurants may create coupon codes with validity dates and discount rules.

Restaurant managers should be able to display a summary of the last month, including total revenue, item-wise revenue, the highest-value order, and the most frequently ordering customer.

This project will be submitted as one final deliverable. The final submission must include the database design, SQL files, report, and working application.

2 Part 1 – Database Design

You are required to:

1. Design the Entity-Relationship (ER) Diagram using proper notation (as shown in class). Hand-drawn diagrams or auto-generated MySQL ERs will not be accepted.
2. Create a DDL SQL file to define the schema.
3. Create a DML SQL file to populate the schema with meaningful sample data.
4. Write a report including:
   - ER diagram
   - List of functional dependencies
   - Explanation of normalization (up to 3NF)
   - Key design decisions and constraints explanations

2.1 Entities and Relations (Minimum Required)

You are required to create an ER diagram representing at least the following entities:

User: A platform user who can be either a customer or a restaurant manager. Each user must have a username and password to log into the system. A user may also have multiple addresses and phone numbers.

Restaurant: Each restaurant has a name, cuisine type, address including city information, and a menu. A restaurant is managed by a user with restaurant manager privileges. A restaurant can also be associated with multiple keywords defined by the manager, which are used for searching and browsing.

MenuItem: Food or drink items offered by a restaurant. Each menu item must have a name, description, price, and image. Menu items belong to a restaurant and may also belong to a menu category.

MenuCategory: A category used to organize menu items, such as starters, main dishes, desserts, drinks, or special offers. Each restaurant may define its own menu categories, and each menu item should be assigned to one category.

Order: Represents an order placed by a customer and fulfilled by a restaurant. An order can contain multiple menu items with specified quantities. It has a status representing its progress, such as preparing, sent, or accepted.

Coupon: A promotional discount code created by a restaurant manager. Each coupon should include a code, discount amount or percentage, validity period, and usage status. Customers may apply a valid coupon while placing an order.

Rating: After an order is accepted, customers can leave a rating from 1 to 5 and an optional comment. Average rating values directly influence restaurant visibility in search results.

The relationships in your ER diagram should include at least the following:

- A Customer can place multiple Orders.
- Each Order includes one or more MenuItems from a single Restaurant.
- A Restaurant can have many MenuItems and receive many Orders.
- A Restaurant is managed by a RestaurantManager.
- A RestaurantManager can manage one or more Restaurants.
- A Restaurant can define multiple MenuCategories.
- Each MenuItem belongs to one MenuCategory.
- A Restaurant can define multiple Coupons.
- A Customer may apply a valid Coupon to an Order.
- A Customer can leave ratings and optional comments for Restaurants they have ordered from.
- Each Order is linked to order-related details such as status and timestamp, even if actual delivery is not implemented.

Important Note 1: For drawing the ER diagram, you may use tools such as Microsoft Excel, Draw.io, LucidChart, SmartDraw, or any other suitable diagramming software. Hand-drawn ER diagrams will not be accepted.

Important Note 2: Your ER diagram style is required to follow the notation taught by the professor during lectures. Other ER diagram styles will not be accepted.

Important Note 3: Automatic ER diagram generation through MySQL is prohibited.

2.2 Data Definition Language (DDL.sql) SQL Code

Write SQL scripts to create all tables defined in your ER diagram:

- Include Primary Keys (PK), Foreign Keys (FK), Unique constraints, NOT NULL, CHECK conditions.
- Establish foreign keys (FK) to maintain relational integrity.
- Implement necessary data types, constraints, and indexes.
- Ensure referential integrity among tables.

2.3 Data Modification Language — DML.sql

You must write DML SQL scripts to insert meaningful sample data into the tables. Your database must include at least the following records:

- 5 Restaurants with related keywords
- 3 Restaurant Managers
- 5 Customers
- 15 Menu Items
- 5 Menu Categories
- 5 Coupons
- 10 Orders placed by different customers
- Each Order must include one or more MenuItems with quantities
- 10 Ratings in total, distributed across restaurants

Your sample data should be realistic and consistent with your ER diagram. For example, menu items should belong to existing restaurants and categories, orders should include items from a single restaurant, and coupons should only be valid for the restaurants that created them.

Tip: You may use real restaurant or menu examples for inspiration, but you must not copy data directly from real platforms.

3 Full Stack Application

You will develop a full-stack application that uses your MySQL database to provide an interactive interface for the Online Food Ordering System. The application must allow users to interact with the system through a Java-based user interface and must connect to a backend implemented with Java Spring.

3.1 Requirements

- **Backend:** The backend must be implemented using Java Spring / Spring Boot. It should handle application logic, user operations, restaurant operations, order processing, and database communication.
- **Database:** The system must use MySQL as the relational database management system.
- **Database Connection:** The backend must connect to MySQL using Java database connectivity methods such as JDBC or Spring-supported database connection mechanisms.

  ORM frameworks that automatically generate SQL queries are not allowed.
- **User Interface:** The user interface must be implemented using Java-based UI technologies. Students may use Java-supported frontend approaches such as JavaFX, Swing, or a Java web UI approach compatible with Spring.

  The interface must be fully functional and must allow users to access all required system features.

  Creativity Note: You are encouraged to be as creative as you want with the user interface design. Creative, user-friendly, and visually appealing interfaces may receive extra points during evaluation.
- **SQL Queries:** All SQL operations, including `INSERT`, `DELETE`, `UPDATE`, and `SELECT`, must be manually written by students. Automatically generated queries, ORM-based query generation, and non-standard shortcuts are not allowed.
- **General Rule:** The application must clearly separate database operations, backend logic, and user interface components. The program must be runnable and testable during the demo session.

3.2 Functional Requirements

The system will support two main types of users: Restaurant Managers and Customers.

Restaurant Managers will be able to define restaurant profiles, manage menus, organize menu items into categories, define discounts or coupons, accept incoming orders, and view sales statistics.

Customers will be able to search for restaurants, browse menus, add items to an order, apply valid coupons, submit orders, track order status, and rate restaurants after their orders are accepted.

3.2.1 User Types

1. Restaurant Manager Users

- Can create and update restaurant profiles, including restaurant name, city, cuisine type, address, and keywords.
- Can define keywords to improve restaurant visibility in search results.
- Must create and manage a menu.
- Can organize menu items using Menu Categories such as starters, main dishes, desserts, and drinks.
- Can add, update, and delete menu items. Each menu item must include name, description, image, price, and category.
- Can define time-limited discounts or Coupons for their restaurant.
- Can manually accept incoming order requests.
- Can view average restaurant rating.
- Average rating out of 5 is shown only after receiving at least 10 ratings. Restaurants with fewer than 10 ratings will appear in search results with a rating of 0 and a "New" label.
- Can view sales statistics for the past month, including:
  - Total revenue and total number of orders,
  - Total quantity sold and revenue per menu item,
  - The customer who placed the most orders in the last month,
  - The customer with the highest-value order in the last month, including order details.
  - The most frequently ordered menu item,
  - The menu category that generated the highest revenue.
  - Total discount amount applied through coupons.

2. Customer Users

- Can register and log in to the system.
- Can search for restaurants based on keywords.
- Can only view and order from restaurants located in the same city as the customer.
- Can browse restaurant menus by menu category.
- Can add multiple menu items and quantities to an order.
- Can apply a valid coupon before finalizing the order.
- Can finalize the order and send an order request to the restaurant.
- Can track the status of their order.
- After the restaurant accepts the order, they can leave a rating and optional comment within 24 hours.

Search results should be ranked by keyword match and average rating, if available. If a restaurant has fewer than 10 ratings, its average rating will not be displayed, and it will be marked as "New".

3.2.2 Order Process Flow

The system must support the following order stages:

- **Order Preparation:** Customer builds the order by selecting menu items and quantities.
- **Order Sent:** Customer confirms the order and sends it to the restaurant.
- **Restaurant Accepted:** Restaurant manager reviews and accepts the order.

Each stage must be represented and tracked in the database using order status and timestamp information.

3.3 Excluded Features

The following features are outside the scope of this project:

- Online payment processing
- Real delivery tracking
- Courier assignment
- External restaurant platform integration

3.4 Non-Functional Requirements

- All SQL queries must be clearly written and documented.
- ORM libraries and automated query builders are not allowed.
- The application must handle errors and exceptions properly.
- The backend code must be organized, readable, and modular.
- The Java UI must provide access to all required functionalities.
- A final demo must be presented showing how the application works and how the main SQL queries are executed.

4 Submission Guidelines

4.1 General Guidelines

Guidelines to ensure a successful project completion:

- Ensure clarity and correctness in the ER diagram, representing all necessary entities and relationships. You are required to use the symbols as presented in the class, otherwise you get penalty.
- DDL SQL code **should** accurately create the database schema based on the ER diagram.
- Use appropriate data types, primary and foreign keys, constraints, and naming conventions.
- You **should** write your own SQL queries manually. You **are NOT allowed** to use a framework that automatically generates SQL statements and not allowed to use frameworks that might abstract the database interaction such as ORM frameworks.
- Your program is **required** to handle all the exceptions and errors.

5 Submission Guidelines

5.1 General Guidelines

- Ensure clarity and correctness in the ER diagram, representing all necessary entities and relationships.
- **You are required to use the ER diagram symbols presented in the class. Otherwise, a penalty may be applied.**
- The DDL SQL code must accurately create the database schema based on the ER diagram.
- Use appropriate data types, primary keys, foreign keys, constraints, and naming conventions.
- The DML SQL code must insert meaningful and consistent sample data into the database.
- **All SQL queries must be written manually.**
- **You are not allowed to use ORM frameworks or any tool/framework that automatically generates SQL statements or abstracts database interaction.**
- The program must properly handle exceptions and errors.
- The submitted application must be runnable and must include all required functionalities.

5.2 Final Submission Guidelines and Requirements

Final Project Deadline: 30th of May 2026, 11:59 PM

Please compress all your files and submit a single `.zip` file.

The name of your zip file should follow this format:

`CS202 Group[id of your group] Project.zip`

Example:

`CS202 Group61 Project.zip`

Inside the zip file, you must provide the following:

- `Group[id of your group] Project Report.pdf`
- `DDL.sql`
- `DML.sql`
- Complete Java Spring / Spring Boot backend source code
- Complete Java UI source code
- Any additional files required to run the program
- A README file explaining how to set up and run the project

Your report must include:

- Group ID
- Names and student IDs of all group members
- ER diagram
- List of functional dependencies
- Explanation of normalization up to 3NF
- Key design decisions and constraint explanations
- Description of the main SQL queries used in the application
- Screenshots or explanations of the implemented functionalities

- The ER diagram may also be submitted separately in PDF, JPEG, or PNG format inside the zip file.
- **Automatic ER diagram generation through MySQL is prohibited.**
- **The zip file must include the working version of the full program. Otherwise, the project may receive zero.**
- If you revise your ER diagram during implementation, you must include the final version in your report and clearly explain the reason for the design changes.
- Failing to comply with these guidelines may result in a penalty.